Desktop systems

- Personal computers -- desktop and laptops
- User convinence and Responsiveness
- Examples: Windows, Mac, Linux, few UNIX, ...

Handheld systems

- OS installed on handheld devices like mobiles, PDAs, iPODs, etc.
- Challenges:
  - Small screen size
  - Low end processors
  - Less RAM size
  - Battery powered
- Examples: Symbian, iOS, Linux, PalmOS, WindowsCE, etc.

Realtime systems

- The OS in which accuracy of results depends on accuracy of the computation as well as time duration in which results are produced, is called as "RTOS".
- If results are not produced within certain time (deadline), catastrophic effects may occur.
- These OS ensure that tasks will be completed in a definite time duration.
- Time from the arrival of interrupt till begin handling of the interrupt is called as "Interrupt Latency".
- RTOS have very small and fixed interrupt latencies.
- RTOS Examples: uC-OS, VxWorks, pSOS, RTLinux, FreeRTOS, etc.

Distributed systems

- Multiple computers connected together in a close network is called as "distributed system".
- Its advantages are high availability (24x7), high scalability (many clients, huge data), fault tolerance (any computer may fail).
- The requests are redirected to the computer having less load using "load balancing" techniques.
- The set of computers connected together for a certain task is called as "cluster". Examples: Linux.

Kernel

- The core OS that does basic minimal functionalities of the kernel is referred as "kernel".
- These basic functionalities are
  - CPU scheduling
  - Process Management
  - Memory Management
  - File & IO Management
  - Hardware abstraction layer
- Kernel has four types (as per architecture)
  - Monolithic kernel
  - Micro kernel
  - Modular kernel

    - Hybrid kernel

Monolithic kernel

- All source files are compiled into a single binary kernel image.
- This is kernel is very fast (all components in the same file).
- However if any component fails, whole OS (kernel) crash.
- Modifying any component needs to rebuild whole kernel and redeploy it.
- e.g. Linux (vmlinuz), UNIX.

Microlithic kernel

- Kernel is very small including minimal functionalities. So referred as micro-kernel.
- All additional functionalities are implemented as independent processes (called as servers).
- All these servers and kernel communicate with each other by message passing. This reduce performance.
- If any component fails, the rest of system can continue to function. So this kernel is robust.
- Modifying any component needs to rebuild and redeploy only that component.
- e.g. Symbian.

Modular kernel

- Modular kernel is small. Additional functionalities are implemented as dynamically loadable modules (.sys/.ko).
- These modules are loaded into the kernel process at runtime and execute.
- This kernel execute faster, as all modules are part of same kernel process.
- If any component fails, whole OS (kernel) crash.
- Modifying any component needs to rebuild and redeploy only that component.
- e.g. Windows (ntoskrnl.exe)

Hybrid kernel

- This kernel is made by combining source code of multiple kernels.
- e.g. Mac OS X kernel is made up of MACH kernel and BSD UNIX kernel.

Linux kernel

- Linux kernel has static and dynamic components.
- Static components are
    - Scheduler
    - Process management
    - Memory management
    - IO subsystem (core)
    - System calls
- Dynamic components are
    - File systems (like ext3, ext4, xfs)
    - Device drivers
- Static components are compiled into the kernel binary image. They are kernel components. The kernel image is /boot/vmlinuz.

- Dynamic components are compiled into kernel objects (*.ko files). They are non-kernel components. They are located in /lib/modules/kernel-version.

Program

- Set of instructions given to the computer --> Executable file.
- Program --> Sectioned binary --> "objdump" & "readelf".
    - Exe header --> Magic number, Address of entry-point function, Information about all sections. (objdump -h program.out)
    - Text --> Machine level code (objdump -S program.out)
    - Data --> Global and Static variables (Initialized)
    - BSS --> Global and Static variables (Uninitialized)
    - RoData --> String constants
    - Symbol Table --> Information about the symbols (Name, Size, section, Flags, Address) (objdump -t program.out)
- Program (Executable File) Format
    - Windows -- PE
    - Linux -- ELF
- Program are stored on disk (storage).

Process

- Progam under execution
- Process execute in RAM.
- Process control block contains information about the process (required for the execution of process).
    - Process id
    - Exit status
        - 0 - Indicate successful execution
        - Non-zero - Indicate failure
    - Scheduling information (State, Priority, Sched algorithm, Time, ...)
    - Memory information (Base & Limit, Segment table, or Page table)
    - File information (Open files, Current directory, ...)
    - IPC information (Signals, ...)
    - Execution context (Values of CPU registers)
    - Kernel stack
- PCB is also called as process descriptor (PD), uarea (UNIX), or task_struct (Linux).
- In Linux size of task_struct is approx 4KB

Redirection

- for every command input is taken from terminal, output is printed on terminal and error is also printed on terminal
- Standard streams (by defualt for every process, three files are opened)
    - stdin
    - stdout
    - stderr
- There are three types of redirections

- input redirection
    - input will be taken from file instead of stdin
    - to do input redirection '<' symbol is used
    - command < file
- output redirection
    - output will be written into file instead of stdout
    - to do output redirection '>' or '>>' symbol is used
    - command > file
        - older content of file will be over written
    - command >> file
        - content will be appneded into file at the end
- error redirection
    - error will be written into file instead of stderr
    - to do output redirection '2>' or '2>>' symbol is used
    - command 2> file
        - older content of file will be over written
    - command 2>> file
        - content will be appneded into file at the end

Pipe

- Using pipe, we can redirect output of any command to the input of any other command.
- Two processes are connected using pipe operator (|).
- Two processes runs simultaneously and are automatically rescheduled as data flows between them.
- If you don't use pipes, you must use several steps to do single task.
- command1 | command2
    - output of command1 will be given as input to command 2
- E.g.
    - who | wc

Linux commands

- cd
    - cd ~ - change working directory to home directory
    - cd - - change working directory to old working directory
    - cd .. - change working directory to parent directory
- ls
    - ls - list the contents of present working directory
    - ls path - list the contents of given path
- touch
    - if file does not exist, empty file is created

    - if file exist, timestamp of that file is changed
- pwd
    - pwd - print current working directory
- mkdir
    - mkdir dir - create directory
    - mkdir -p a/b/c - create nested directories
- rmdir
    - rmdir dir - remove empty directory
- rm
    - rm file - remove file
    - rm -r dir - remove directory with contents
    - rm -rf dir - force remove directory
- cat
    - cat file - display file content
    - cat > file - create new file
    - cat >> file - append to file
- head
    - head file - display first 10 lines
    - head -5 file - display first 5 lines
- tail
    - tail file - display last 10 lines
    - tail -4 file - display last 4 lines
- sort
    - sort file - sort the content by alphabetically
    - sort -n file - sort the content by their value
    - sort command do not modify file content
- uniq
    - uniq file - display contents uniquely (truncate duplicate)
        - truncate duplicate content if it is consecutive
- rev filepath
    - Print each line reversed.
    - File contents are not modified.
- tac filepath

- Print all lines in reverse order. The first line printed at last, while last line printed first.

Directory commands

- pwd -- print present working directory (current directory)
- cd -- change directory (syntax> cd dirpath)
- ls - list directory contents (syntax> ls dirpath)
- mkdir -- make directory (syntax> mkdir dirpath)
- rmdir -- remove empty directory (syntax> rmdir dirpath)
- cd
  - cd ~ - change working directory to home directory
  - cd - - change working directory to old working directory
  - cd .. - change working directory to parent directory

File commands

- cat
  - cat > filepath <-- create new file
  - cat filepath <-- display file contents
- rm
  - rm filepath <-- delete given file
  - rm -r dirpath <-- delete dir with all contents
- mv
  - mv filepath destdirpath <-- move given file into given dest directory
  - mv dirpath destdirpath <-- move given dir into given dest directory
  - mv oldname newname <-- rename given file
- cp
  - cp filename newfilename <-- copy file with new name/path.
  - cp filepath destdirpath <-- copy file into given dest dir with same name.
  - cp -r dirpath destdirpath <-- copy file into given dest dir with same name.
- Special Characters
  - ~ - home directory
  - . - current directory
  - .. - parent directory
  - / - root directory separator
  - \ - escape character
  - matches multiple characters (wildcard)
  - ? - matches single character

- redirect output (overwrite)
- redirect output (append)

- < - redirect input